D

Dumbhanengue ERP

14 — Go-Live : opérations externes (checklist)

Version 1.1.1+3 · 16 août 2026

Dumbhanengue ERP

Table des matières

0. État actuel 3
1. Secrets GitHub (Settings → Secrets and variables → Actions) 3
2. Supabase (projet de production) 3
3. Google Play Console 5
4. Apple (iOS) 6
5. Avant le premier tag 6
6. Après le premier upload 6

> Ce chapitre couvre tout ce qui n'est pas dans le dépôt — les étapes qui exigent les consoles (Google Play, Apple Developer, Supabase) et les secrets CI. Le code de release est prêt (workflows + Fastlane) ; ces étapes sont manuelles et ponctuelles.

0. État actuel

- Keystore d'upload généré (android/app/upload-keystore.jks, gitignoré)

+ android/key.properties — signature Android prête.

- release.yml (Android → Play *Internal testing*) et ios.yml

(iOS → TestFlight) prêts ; déclenchés par les tags v*.

- Secrets CI à configurer ; comptes des stores ; activations des consoles.

1. Secrets GitHub (Settings → Secrets and variables → Actions)

Secret	Source
SUPABASE_URL_PROD	URL du projet de production (dashboard Supabase)
SUPABASE_ANON_KEY_PROD	clé anon du projet de production
SENTRY_DSN (facultatif)	projet Sentry
KEYSTORE_BASE64	base64 -w0 android/app/upload-keystore.jks (déjà à android/app/upload-keystore.jks.b64 — copiez le contenu, puis supprimez le fichier)
KEYSTORE_PASSWORD	storePassword de android/key.properties (PKCS12 : identique à keyPassword)
KEY_ALIAS	keyAlias de android/key.properties (défaut upload)
PLAY_SERVICE_ACCOUNT_JSON	compte de service Play Console (Google Play Developer API) — voir §3
APPLE_ISSUER_ID	App Store Connect → Users and Access → Integrations → App Store Connect API → Issuer ID
APPLE_KEY_ID	id de la clé créée sur le même écran
APPLE_KEY_CONTENT	contenu base64 du fichier .p8 de cette clé API
APPLE_TEAM_ID	Apple Developer → Membership → Team ID

2. Supabase (projet de production)

1. Confirmation d'e-mail ON — Authentication → Providers → Email →

« Confirm email » activé (atténue l'auto-inscription ; prérequis de sécurité).

1. 2FA sur le compte super admin — activez-la sur le compte e-mail des

super_admins (ou idéalement TOTP dans Supabase lui-même quand c'est disponible).

1. `password_min_length = 8` — Authentication → Settings (s'aligne sur le client).
2. Appliquez l'upgrade consolidée au projet de production : SQL Editor ←

contenu de supabase/upgrade.sql (amène une base EXISTANTE à l'état actuel — remplace les anciennes migrations 002..011 ; 100 % idempotent, sûr à ré-exécuter). Pour les projets neufs, supabase/schema.sql seul suffit.

1. Limites d'e-mails (dev) : gardez les défauts. Sur le plan gratuit,

l'envoi est plafonné (~2 e-mails/heure) — quand la quota est épuisée, GoTrue répond 429 « email rate limit exceeded » et l'app affiche le message spécifique *« Trop de tentatives consécutives. Attendez environ 1 heure et réessayez. »* (clé loginRateLimitError — l'inscription reprend à la fenêtre suivante, aucune action requise ; l'enregistrement des employés n'utilise pas d'e-mail — la RPC create_employee_account).

1. SMTP personnalisé en production (plan : Resend + domaine propre) : le

tier gratuit plafonne aussi les e-mails de confirmation (2/heure et 4/jour) — inacceptable en production avec de vraies inscriptions. Avant le go-live :

1. Resend → ajoutez votre propre domaine (ex. dumbhanengue.com) et

terminez la vérification DNS (SPF + DKIM + DMARC). Le domaine de test onboarding@resend.dev ne délivre qu'à l'e-mail du compte — pas utilisable pour de vrais expéditeurs.

1. Resend → créez une clé API envoi uniquement (re_...).
2. Supabase → Authentication → Emails → SMTP Settings :

Champ	Valeur
Host	smtp.resend.com
Port	587
Username	resend (fixe — la clé API va dans Password)
Password	la clé API (re_...)
From email	no-reply@dumbhanengue.com
From name	Dumbhanengue
Admin email	e-mail du super admin (récupération)

1. Alternative programmatique (Management API) : `PATCH

/v1/projects/{ref}/config/auth avec smtp_host, smtp_port, smtp_user (**resend**), smtp_pass (la clé API), smtp_sender_email, smtp_sender_name et smtp_admin_email (jeton sbp_… depuis Settings → Access Tokens). **Le PATCH remplace toute la config** — envoyez tous les champs smtp_*` d'un coup (validé Aug 8 : un PATCH partiel a effacé le SMTP).

1. Test : déclenchez une inscription et confirmez que l'e-mail de

confirmation est délivré ; la clé Resend peut aussi être testée avec un POST direct vers https://api.resend.com/emails.

> Décision enregistrée (Aug 8) : dumbhanengue.com **n'est pas enregistré (RDAP 404 — domaine disponible) et l'équipe a choisi de ne pas acheter de domaine pour l'instant** — l'e-mail de confirmation continue via le SMTP Resend avec le domaine de test onboarding@resend.dev, qui **ne délivre qu'à l'e-mail du compte Resend** (jalaneoanalista@gmail.com) ; tout autre e-mail d'une inscription reçoit un 403. Implications pour le test dev :
>
> - Seules les inscriptions avec jalaneoanalista@gmail.com peuvent être confirmées ; les autres e-mails ne reçoivent jamais le lien (l'app reste sur l'écran de confirmation).
> - Alternative de test : mailer_autoconfirm = true en dev (les nouvelles inscriptions se confirment instantanément, sans e-mail) — utilisée dans les tests E2E.
>
> **Important (mythe démystifié) : payer un plan Supabase ne résout PAS la délivrance des e-mails.** Le service d'e-mail intégré de Supabase est sévèrement limité même sur les plans payants (~2 e-mails/heure ; tier gratuit restreint aux membres de l'organisation) — la recommandation officielle est un SMTP externe (ce que nous utilisons déjà : Resend). Le vrai déblocage ne coûte que l'enregistrement du domaine (~8–12 US$/an) : le tier gratuit de Resend permet déjà 1 domaine vérifié (SPF/DKIM/DMARC) avec 3 000 e-mails/mois (100/jour) — suffisant pour le go-live d'une petite boutique, sans coût côté Supabase.

3. Google Play Console

1. Créez l'app avec l'Application ID `com.dumbhanengue.dumbhanengue`

(immuable après le premier upload).

1. Activez Play App Signing et enregistrez le SHA-256 de la clé d'upload

(depuis keytool -list -v -keystore upload-keystore.jks).

1. Créez le compte de service avec la permission d'upload (Play Console →

Setup → API access) et stockez le JSON dans le secret PLAY_SERVICE_ACCOUNT_JSON.

1. Livrez la première release : git tag v1.0.0 && git push --tags → la CI

construit et uploade vers le track Internal testing.

1. Assets du store : icône (déjà générée), image de fonctionnalité, captures

d'écran, description, politique de confidentialité + CGU (liens obligatoires).

4. Apple (iOS)

1. Apple Developer Program actif + App ID com.dumbhanengue.dumbhanengue.
2. Configurez la signature automatique dans Xcode (équipe) OU utilisez match

(alternative documentée dans la lane ios beta).

1. Créez la clé API App Store Connect (fichier .p8) et ajoutez les 3 secrets

Apple (§1). La lane fastlane ios beta valide d'abord le build sans
signature (--no-codesign), puis signe et uploade.

1. Note : le build iOS exige macOS — la CI utilise macos-14.

Localement (Windows), l'IPA ne peut pas être validé ; la validation se
produit dans le workflow.

5. Avant le premier tag

- [ ] Tous les secrets §1 remplis
- [ ] Supabase : confirmation d'e-mail ON + 2FA + migration 008 appliquée
- [ ] SMTP personnalisé sur le Supabase de production (Resend + domaine propre) — §2.6
- [ ] Play App Signing activé + compte de service créé
- [ ] Apple Developer + clés API créées
- [ ] Politique de confidentialité + CGU publiées (liens du store)
- [ ] flutter test vert + sql_check.sh 9/9 (la CI valide à chaque push)

6. Après le premier upload

- [ ] Testez l'APK/AAB Internal testing sur un vrai appareil (propriétaire,

gérant et employé — matrice d'accès)

- [ ] Vérifiez Sentry (si DSN configuré) pour les erreurs de release
- [ ] Promouvez d'Internal → Closed/Open testing après approbation manuelle